ALOUANI Fadi

BTS SIO 1
SLAM
05/11/26

[REACT] TP1

Sommaire

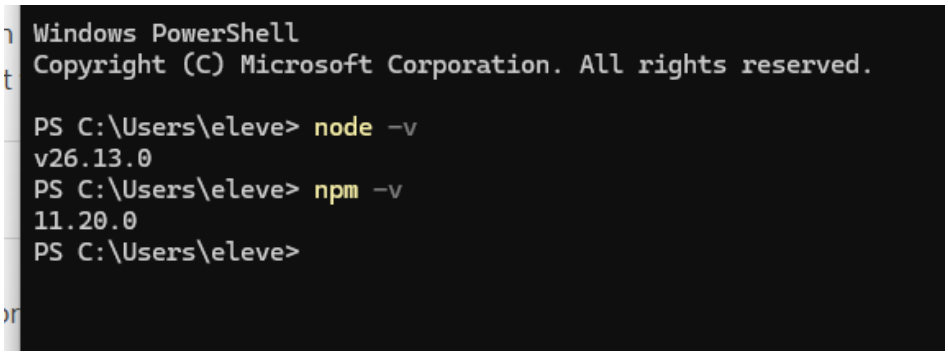
Introduction....1
Partie 1 : Installation de l'environnement....2
1.1 Installer Node.js....2
Question :1....2
Partie 2 : Création du projet....2
Question :2....3
Question :3....3
Partie 3 : Découverte de l'arborescence....4
Question :4....5
Partie 4 : Premier composant en JSX....5
Question :5....8
Question :6....9
Partie 5 : Les props....9
Question :7....12
Partie 6 : L'état avec useState....12
Question 8 :....13
Question 9 :....14
Partie 7 : Afficher une liste....14
Question 10....14
Conclusion....15

Introduction

Dans ce TP, j'ai découvert le fonctionnement de React et appris à créer une première application web. J'ai installé mon environnement de développement, créé un projet avec Vite et découvert les composants, les props et le state. J'ai également appris à afficher une liste dynamique de tickets.

Partie 1 : Installation de l'environnement

1.1 Installer Node.js



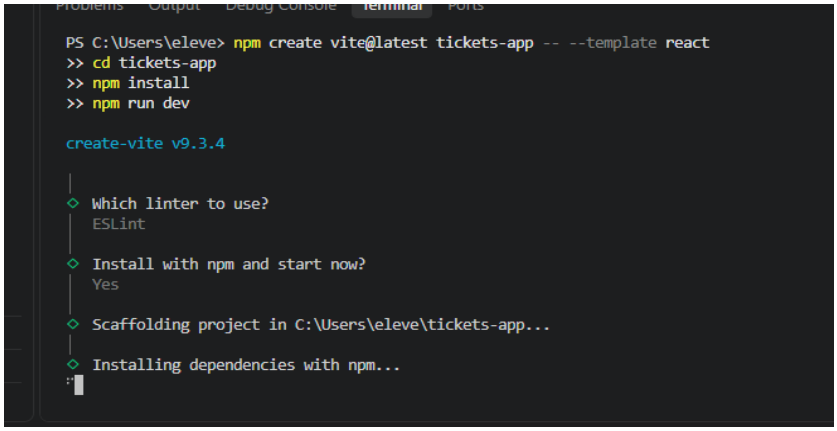
Question :1

Les versions installées sont :

- Node.js : v26.13.0
- npm :11.20.0

npm (Node Package Manager) est le gestionnaire de paquets de Node.js. Il permet d'installer, de gérer et de mettre à jour les bibliothèques et dépendances nécessaires à un projet JavaScript, comme React.

Partie 2 : Création du projet

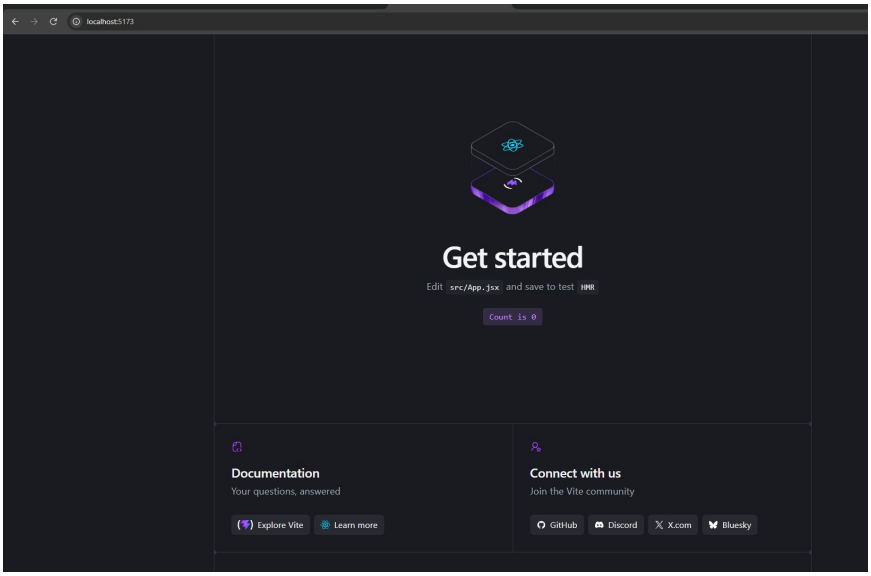


Question :2

1. npm create vite@latest tickets-app -- --template react
Cette commande permet de créer un nouveau projet appelé tickets-app en utilisant Vite et le modèle React.
2. cd tickets-app
Cette commande permet d'entrer dans le dossier tickets-app afin de travailler dans le projet.
3. npm install
Cette commande permet d'installer toutes les dépendances nécessaires au fonctionnement du projet, comme React et Vite.
4. npm run dev
Cette commande démarre le serveur de développement Vite afin de pouvoir visualiser et tester l'application dans un navigateur, généralement à l'adresse http://localhost:5173.

Question :3

Vite est un outil de développement qui permet de créer, lancer et construire des applications web modernes comme les applications React.
Son rôle est de faciliter le développement en lançant un serveur local, en permettant de voir rapidement les modifications du code dans le navigateur et en préparant l'application pour sa mise en production.
Contrairement à un serveur web classique qui sert principalement à envoyer des pages web au navigateur, Vite est spécialement conçu pour faciliter le développement. Il actualise rapidement l'affichage lorsqu'on modifie le code, sans avoir besoin de recharger manuellement toute la page.



Partie 3 : Découverte de l'arborescence

Fichier / dossier	Rôle
`package.json`	Contient les informations du projet, ses dépendances et les commandes disponibles.
`node_modules/`	Contient les bibliothèques et dépendances installées avec npm, comme React et Vite.
`index.html`	Fichier HTML principal qui contient le point d'accroche de l'application React.
`src/main.jsx`	Point d'entrée de React qui permet de démarrer l'application et d'afficher le composant App.
`src/App.jsx`	Composant principal qui contient et organise l'interface de l'application.
`src/assets/`	Dossier qui contient les ressources utilisées dans l'application, comme les images et les icônes.

Question :4

Cette ligne permet de démarrer React et d'afficher l'application dans la page HTML.

`document.getElementById('root')` récupère l'élément HTML qui possède l'identifiant `root`.

`createRoot()` indique à React qu'il doit gérer l'affichage dans cet élément.

Enfin, `.render()` permet d'afficher le composant principal `App` à l'intérieur de cet élément.

Ainsi, React peut afficher et mettre à jour l'interface de l'application dans le navigateur.

Partie 4 : Premier composant en JSX

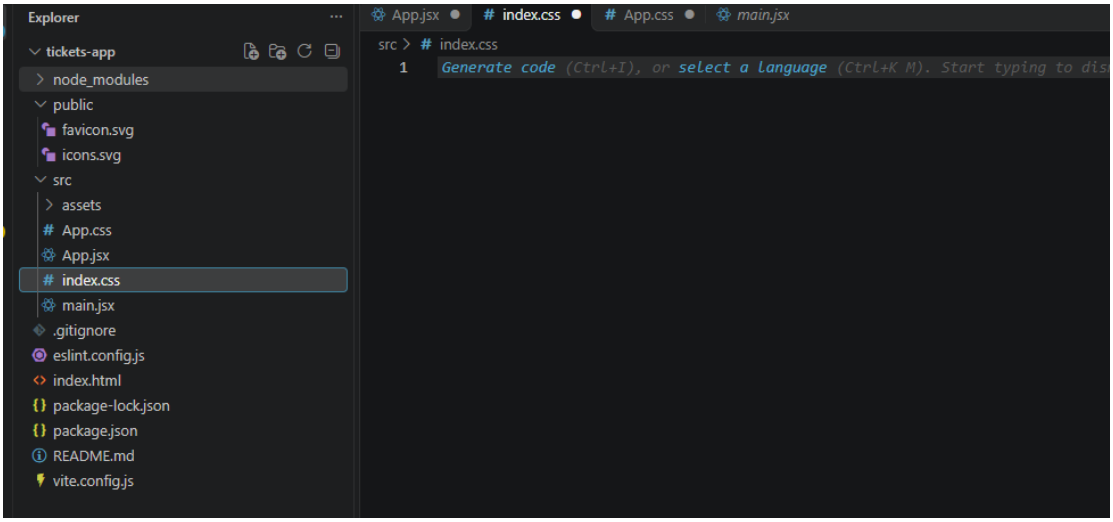
Supprimez le contenu de `App.jsx` et remplacez-le par :

App.jsx

```jsx
function App() {
  return (
    <div>
      <h1>Gestion des tickets</h1>
      <p>Bienvenue sur l'application WebDev Solutions.</p>
    </div>
  );
}

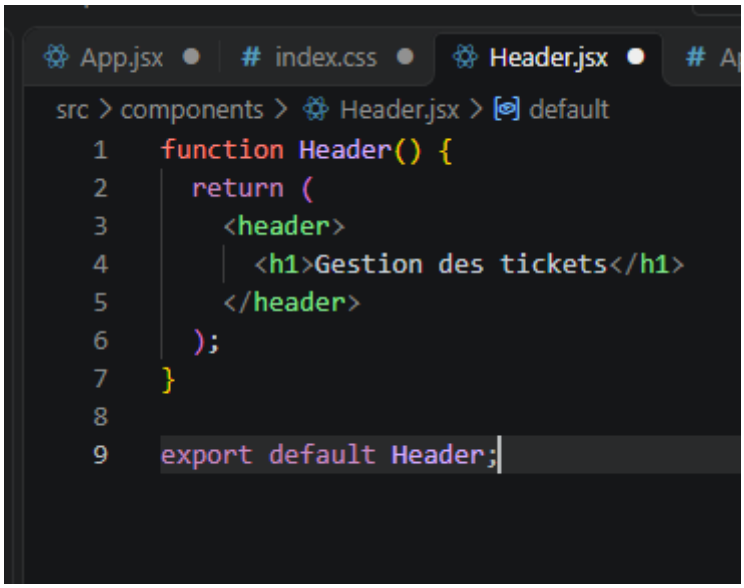
export default App;
```

Videz le fichier `App.css` et `index.css` (ou personnalisez-les).

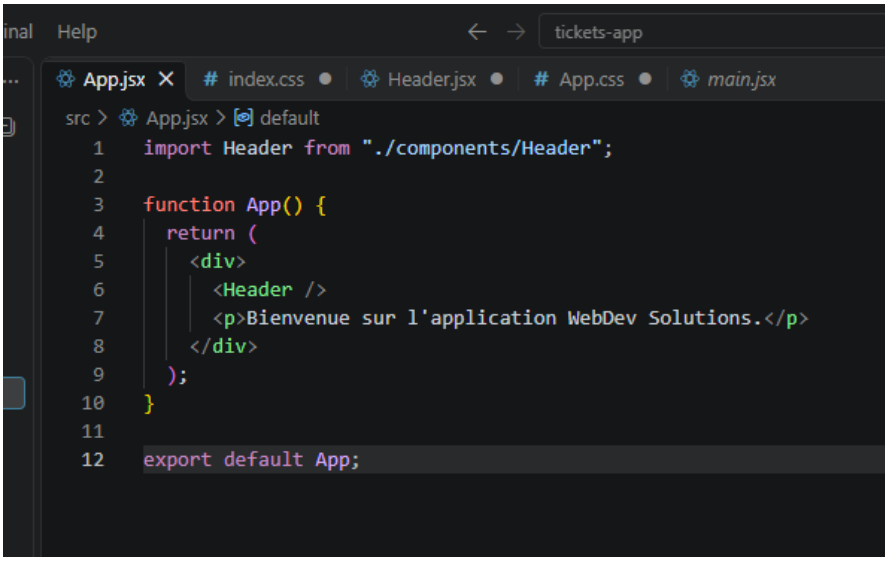


J'ai fait pareil pour app.css

Créez le dossier `src/components/` puis le fichier `Header.jsx` :



Importez et utilisez `<Header />` dans `App.jsx`.



Question :5

Le JSX (JavaScript XML) est une syntaxe utilisée avec React qui permet d'écrire du code ressemblant au HTML directement dans JavaScript. Il permet de créer facilement l'interface d'une application React.
Voici trois différences avec le HTML :

1. class / className : en JSX, on utilise className au lieu de class pour définir une classe CSS.
2. for / htmlFor : en JSX, on utilise htmlFor au lieu de for pour associer un label à un champ de formulaire.
3. Les expressions JavaScript : en JSX, on peut intégrer des variables et des expressions JavaScript entre accolades {}, ce qui n'est pas possible directement de la même manière en HTML.

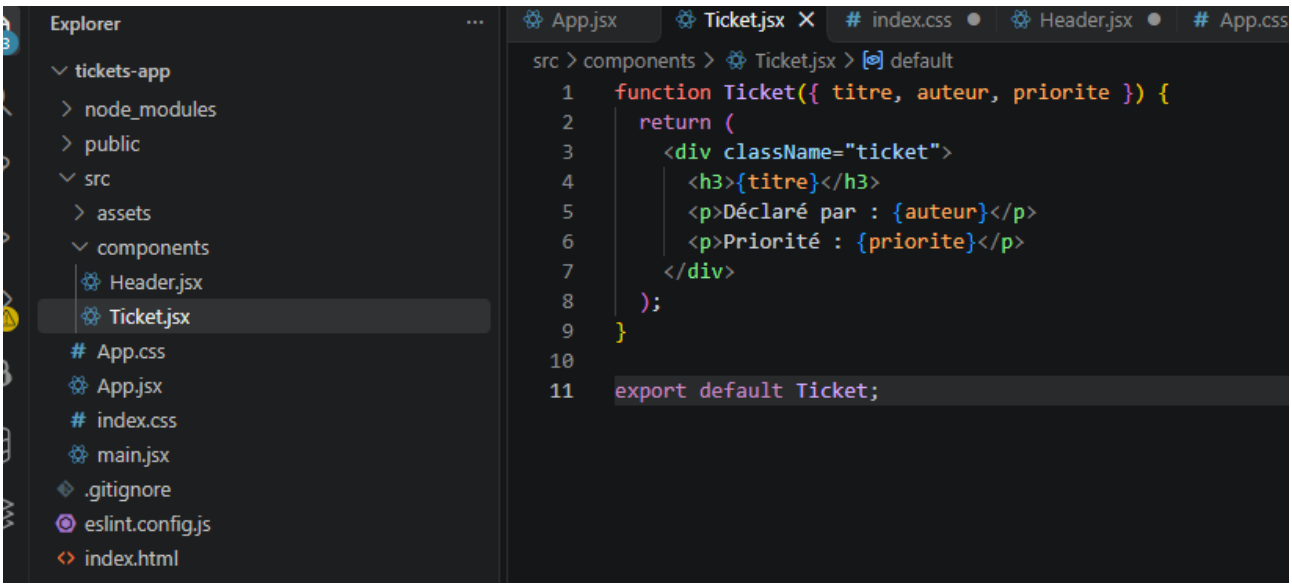
Le JSX permet donc de mélanger facilement la structure de l'interface et les données JavaScript.

Question :6

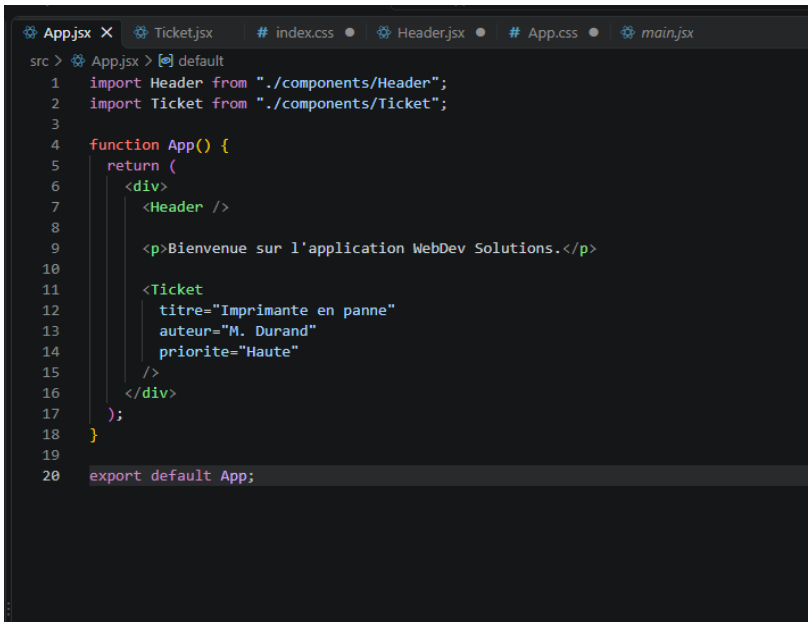
En React, un composant doit retourner un seul élément parent qui contient tous les autres éléments. Cela permet à React de regrouper les différents éléments dans une seule structure JSX et de construire correctement l'interface.
Par exemple, il est possible d'utiliser une balise pour regrouper plusieurs éléments.
Il est également possible d'utiliser un Fragment React (<>...</>) qui permet de regrouper plusieurs éléments sans ajouter de balise HTML supplémentaire dans la page.

Partie 5 : Les props

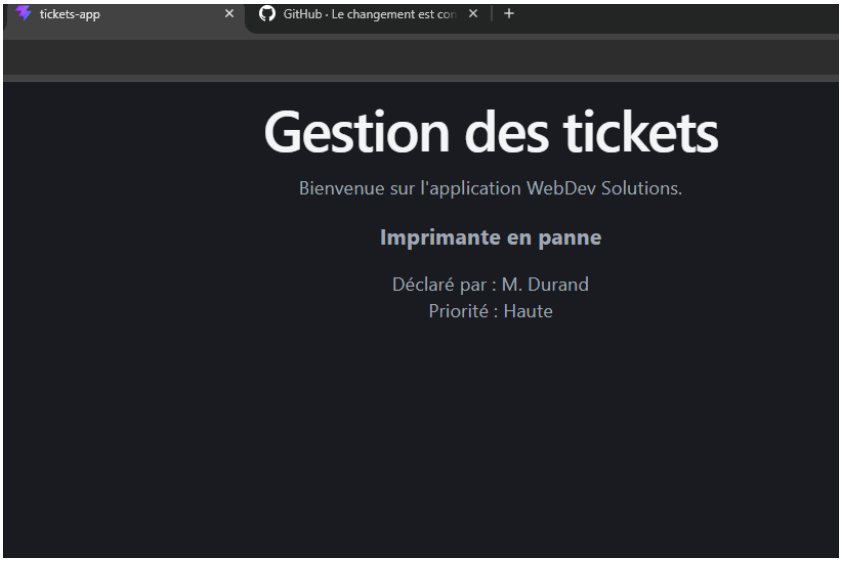
Créez un composant `Ticket.jsx` qui affiche un ticket :



Utilisez-le dans `App.jsx` avec au moins trois tickets différents :

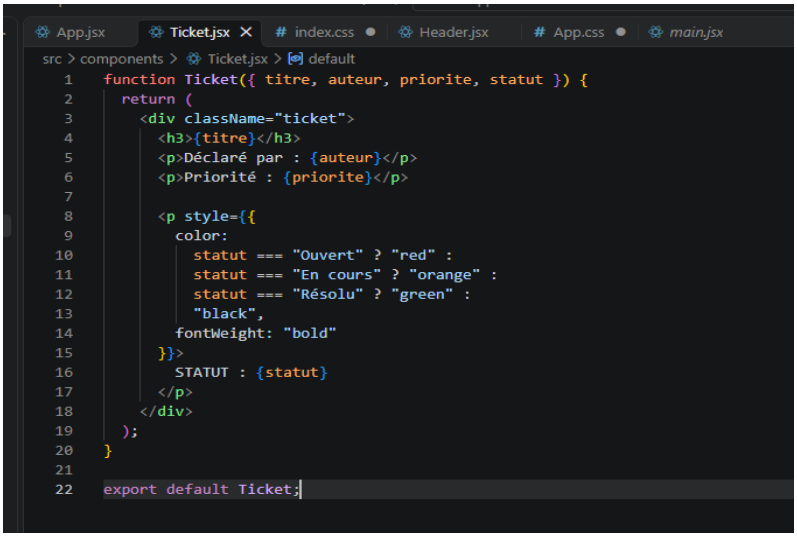


résultat sur le navigateur j'ai ajouter les deux autre tickets ensuite :



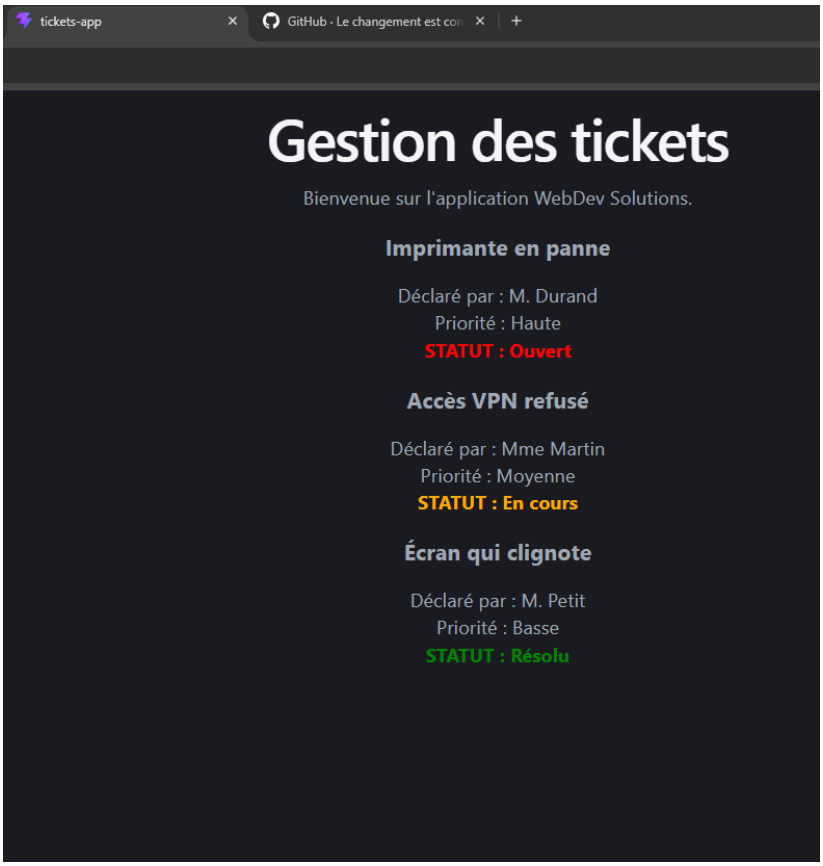
Travail à faire :

•Ajoutez une prop `statut` ("Ouvert", "En cours", "Résolu").
•Affichez le statut en rouge si "Ouvert", orange si "En cours", vert si "Résolu" (rendu conditionnel).





résultat sur le navigateur :



Question :7

Non, les props ne sont pas modifiables directement par le composant qui les reçoit. Elles sont en lecture seule et sont transmises par le composant parent.

Si une valeur doit changer, c'est le composant parent qui doit modifier la donnée et transmettre une nouvelle prop au composant enfant.

Partie 6 : L'état avec `useState`

Travail à faire :

•Ajoutez un bouton « Remettre à zéro ».
•Empêchez le compteur de descendre sous 0 si vous ajoutez un bouton « Annuler ».

```
      <hr />

      <h2>Compteur de tickets traités</h2>

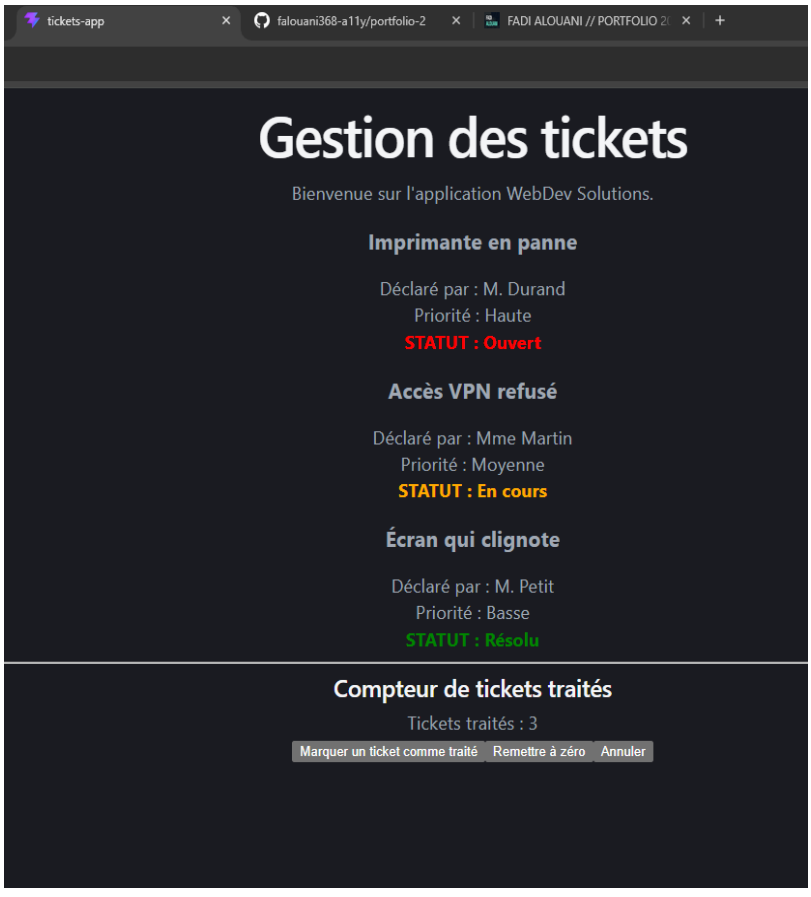
      <p>Tickets traités : {traites}</p>

      <button onClick={() => setTraites(traites + 1)}>
        Marquer un ticket comme traité
      </button>

      <button onClick={() => setTraites(0)}>
        Remettre à zéro
      </button>

      <button onClick={() => setTraites(Math.max(0, traites - 1))}>
        Annuler
      </button>
    </div>
  );
}

export default App;
```



Question 8 :

Une prop est une donnée transmise par un composant parent à un composant enfant. Elle est en lecture seule et ne peut pas être modifiée directement par le composant qui la reçoit.
Un state est une donnée interne à un composant qui peut évoluer au cours de l'utilisation de l'application. Il est géré avec useState et permet à React de mettre à jour automatiquement l'affichage lorsqu'il change.

Question 9 :

Il ne faut pas modifier directement la variable traites car React ne détecte pas correctement ce changement et ne met donc pas automatiquement à jour l'affichage.
Il faut utiliser la fonction setTraites() fournie par useState pour modifier la valeur du state et demander à React de mettre à jour l'interface.

Partie 7 : Afficher une liste

App.jsx | Ticket.jsx | index.css | Header.jsx | App.css | main.jsx

src > App.jsx > ...

```jsx
import { useState } from "react";
import Header from "./components/Header";
import Ticket from "./components/Ticket";

function App() {
  const [traites, setTraites] = useState(0);

  const tickets = [
    {
      id: 1,
      titre: "Imprimante en panne",
      auteur: "M. Durand",
      priorite: "Haute",
      statut: "Ouvert"
    },
    {
      id: 2,
      titre: "Accès VPN refusé",
      auteur: "Mme Martin",
      priorite: "Moyenne",
      statut: "En cours"
    },
    {
      id: 3,
      titre: "Écran qui clignote",
      auteur: "M. Petit",
      priorite: "Basse",
      statut: "Résolu"
    }
  ];

  return (
    <div>
      <Header />

      <p>Bienvenue sur l'application WebDev Solutions.</p>

      <h2>Liste des tickets</h2>

      {tickets.map((t) => (
        <Ticket key={t.id} {...t} />
      ))}

      <hr />

      <h2>Compteur de tickets traités</h2>

      <p>Tickets traités : {traites}</p>

      <button onClick={() => setTraites(traites + 1)}>
        Marquer un ticket comme traité
      </button>
```



```jsx
function App() {
  const tickets = [
    {
      statut: "En cours"
    },
    {
      id: 3,
      titre: "Écran qui clignote",
      auteur: "M. Petit",
      priorite: "Basse",
      statut: "Résolu"
    }
  ];

  return (
    <div>
      <Header />

      <p>Bienvenue sur l'application WebDev Solutions.</p>

      <h2>Liste des tickets</h2>

      {tickets.map((t) => (
        <Ticket key={t.id} {...t} />
      ))}

      <hr />

      <h2>Compteur de tickets traités</h2>

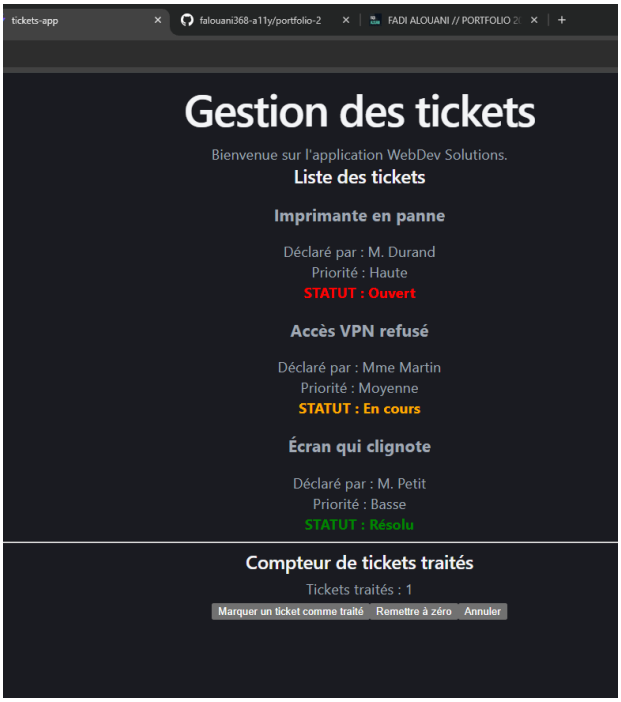
      <p>Tickets traités : {traites}</p>

      <button onClick={() => setTraites(traites + 1)}>
        Marquer un ticket comme traité
      </button>

      <button onClick={() => setTraites(0)}>
        Remettre à zéro
      </button>

      <button onClick={() => setTraites(Math.max(0, traites - 1))}>
        Annuler
      </button>
    </div>
  );
}

export default App;
```



Question 10

La prop key permet à React d'identifier chaque élément d'une liste de manière unique.

Elle aide React à repérer les éléments ajoutés, supprimés ou modifiés afin de mettre à jour l'affichage efficacement.

Si on oublie la prop key, React affiche un avertissement dans la console du navigateur indiquant que chaque élément d'une liste doit posséder une clé unique. L'application peut continuer à fonctionner, mais React risque de moins bien identifier les changements dans la liste.

Conclusion

Ce TP m'a permis de découvrir les bases de React et de mieux comprendre son fonctionnement. J'ai appris à créer des composants, à gérer des données et à rendre une liste dynamique. Ces connaissances me permettront de continuer à développer des applications web avec React.